

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: SATHWIK

Date: 16/7/26

Roll No: 24011101030

Experiment 2

Implementation of a Multi-Layer Perceptron for Multi-Class Image Classification

1. Objective

The objective of this experiment is to implement a Multi-Layer Perceptron using TensorFlow/Keras for multi-class image classification on the Fashion-MNIST dataset. The experiment covers image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization using RandomizedSearchCV together with the SciKeras wrapper.

2. Theory

An MLP consists of an input layer, one or more hidden layers, and an output layer. Each hidden layer applies an affine transformation followed by a non-linear activation function:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

The output layer produces class probabilities via the Softmax function:

$$\hat{y} = \text{Softmax}(z)$$

The network is trained by minimizing the categorical cross-entropy loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

The baseline architecture used in this experiment is:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

- Dataset: Fashion-MNIST
- Training Images: 60,000
- Testing Images: 10,000
- Classes: 10 (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot)
- Image Size: 28×28 , grayscale

Since Dense layers require a one-dimensional feature vector, each 28×28 image is flattened into a 784-length vector ($28 \times 28 = 784$), pixel values are normalized to $[0, 1]$, and class labels are one-hot encoded prior to training.

4. Experimental Procedure

Task 1 – Dataset Exploration: Fashion-MNIST was loaded and its dimensions verified (training set $60000 \times 28 \times 28$, test set $10000 \times 28 \times 28$). Ten sample images and the per-class training distribution were plotted (Section 8, Plots 1–2).

Task 2 – Data Preprocessing: Images were flattened from 28×28 matrices to 784-length vectors and pixel intensities were normalized from the $[0, 255]$ integer range to the $[0, 1]$ floating-point range. Labels were one-hot encoded to 10-dimensional vectors.

Task 3 – Model Construction: A baseline MLP with the architecture $\text{Input}(784) \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$ was constructed in Keras.

Task 4 – Model Training: The baseline model was compiled with the Adam optimizer, categorical cross-entropy loss, and accuracy metric, and trained for 20 epochs with a batch size of 32, using a 20% validation split.

Task 5 – Model Evaluation: Accuracy, macro-averaged precision/recall/F1-score, the confusion matrix, and the full classification report were computed on the held-out test set (Section 9).

5. Source Code

The complete source code and implementation for this experiment can be found in the following GitHub repository:

GitHub Link: <https://github.com/sathwik324/DL-LAB/tree/main/Exp2>

6. Hyperparameter Optimization

Automated hyperparameter optimization was performed using **RandomizedSearchCV** with the SciKeras **KerasClassifier** wrapper and 5-fold cross-validation, over the following search space:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate	0.0, 0.2, 0.5

The full grid contains $3 \times 4 \times 3 \times 4 \times 3 \times 3 \times 3 \times 3 = 3888$ possible combinations. Exhaustive Grid Search over this space (at 5-fold CV) was intractable within the available runtime, so **RandomizedSearchCV** was used with **n_iter=3** sampled configurations, chosen to keep the search runtime manageable on the available compute while still demonstrating the automated search-and-select workflow. The three sampled configurations and their 5-fold mean cross-validation accuracies are shown below.

Rank	Layers	Neurons	LR	Optimizer	Activation	Epochs	Batch	Dropout	Mean CV Acc.
1	3	128	0.001	rmsprop	tanh	30	32	0.2	0.8538
2	1	256	0.001	adam	tanh	20	64	0.2	0.8526
3	3	256	0.01	rmsprop	sigmoid	10	16	0.2	0.8077

The best configuration (Rank 1) was retrained on the full 60,000-image training set and evaluated on the held-out 10,000-image test set (Section 9).

7. Results

Best Hyperparameters:

Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSProp
Activation Function	Tanh
Epochs	30
Dropout	0.2
Cross-validation Accuracy	0.8538
Testing Accuracy	0.8760

Performance Comparison :

Metric	Baseline	Optimized
Accuracy	0.8722	0.8760
Precision (macro)	0.8755	0.8788
Recall (macro)	0.8722	0.8760
F1-score (macro)	0.8725	0.8763
Epochs Trained	20	30

Baseline Classification Report:

Class Name	Precision	Recall	F1-score	Support
T-shirt/top	0.88	0.72	0.79	1000
Trouser	0.99	0.96	0.98	1000
Pullover	0.77	0.81	0.79	1000
Dress	0.89	0.89	0.89	1000
Coat	0.80	0.78	0.79	1000
Sandal	0.97	0.96	0.97	1000
Shirt	0.65	0.71	0.68	1000
Sneaker	0.91	0.97	0.94	1000
Bag	0.91	0.98	0.94	1000
Ankle boot	0.97	0.93	0.95	1000
Accuracy			0.87	10000
Macro Avg	0.88	0.87	0.87	10000
Weighted Avg	0.88	0.87	0.87	10000

Optimized Classification Report:

Class Name	Precision	Recall	F1-score	Support
T-shirt/top	0.89	0.78	0.83	1000
Trouser	0.99	0.97	0.98	1000
Pullover	0.73	0.85	0.79	1000
Dress	0.86	0.90	0.88	1000
Coat	0.80	0.78	0.79	1000
Sandal	0.97	0.95	0.96	1000
Shirt	0.70	0.68	0.69	1000
Sneaker	0.90	0.96	0.93	1000
Bag	0.98	0.97	0.97	1000
Ankle boot	0.97	0.92	0.95	1000
Accuracy			0.88	10000
Macro Avg	0.88	0.88	0.88	10000
Weighted Avg	0.88	0.88	0.88	10000

8. Plots with Inference

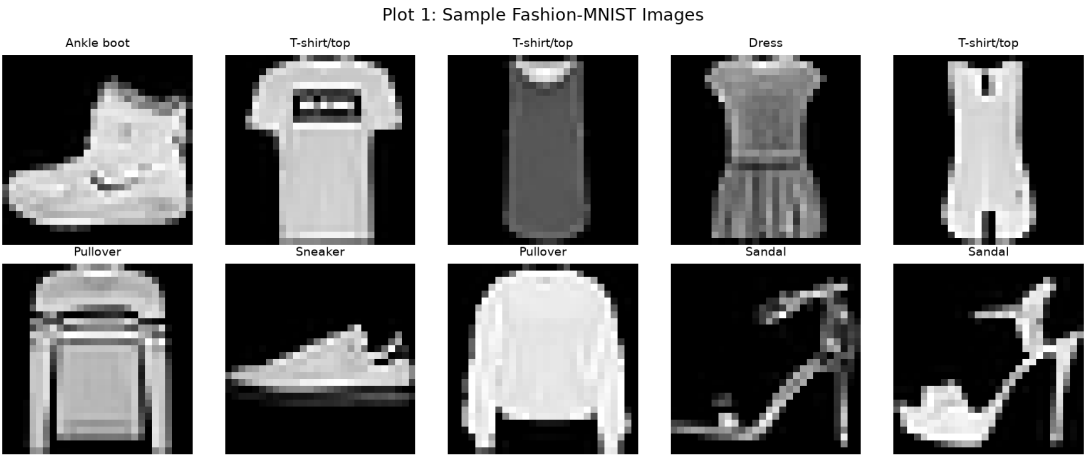


Figure 1: Sample Fashion-MNIST training images.

Inference: The sample grid confirms images are correctly loaded as 28×28 grayscale garments with visibly distinct silhouettes across classes.

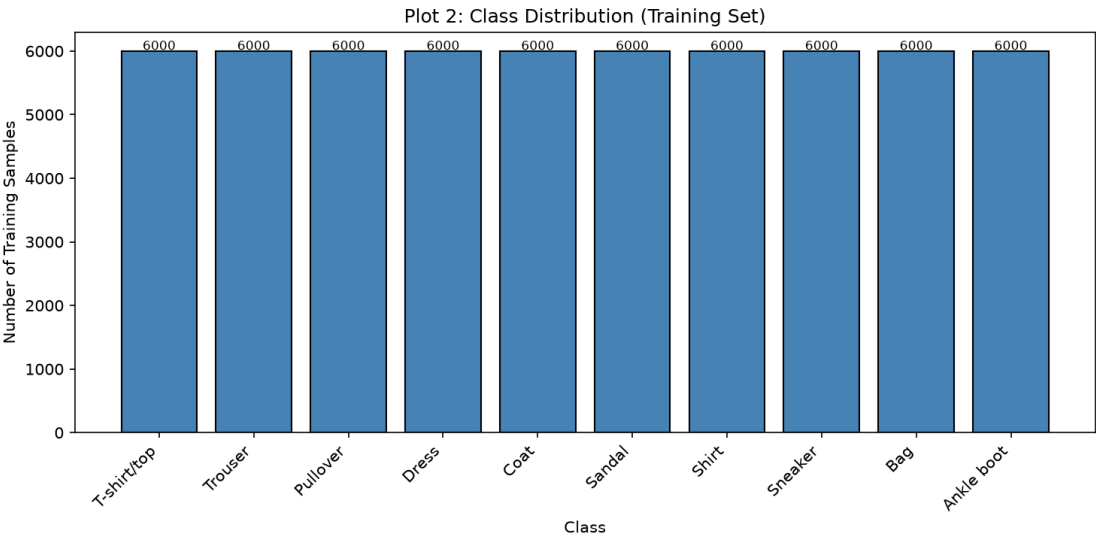


Figure 2: Class distribution of the training set.

Inference: Every class contains exactly 6000 training samples, confirming the dataset is perfectly balanced. This means accuracy is a reliable metric and no class-weighting is required during training.

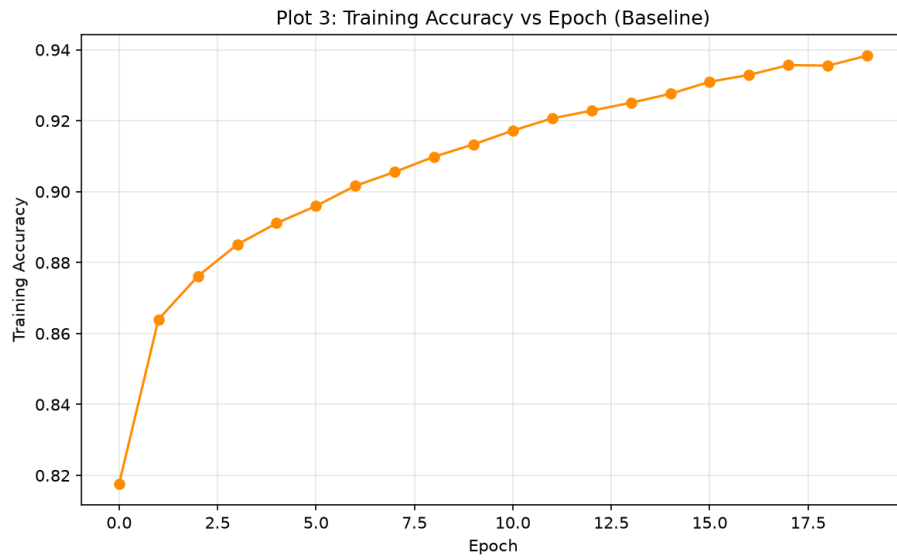


Figure 3: Training accuracy vs. epoch (baseline model).

Inference: Training accuracy rises steadily from about 0.82 to nearly 0.94 across 20 epochs, indicating the baseline MLP is learning the training distribution consistently without stalling early.

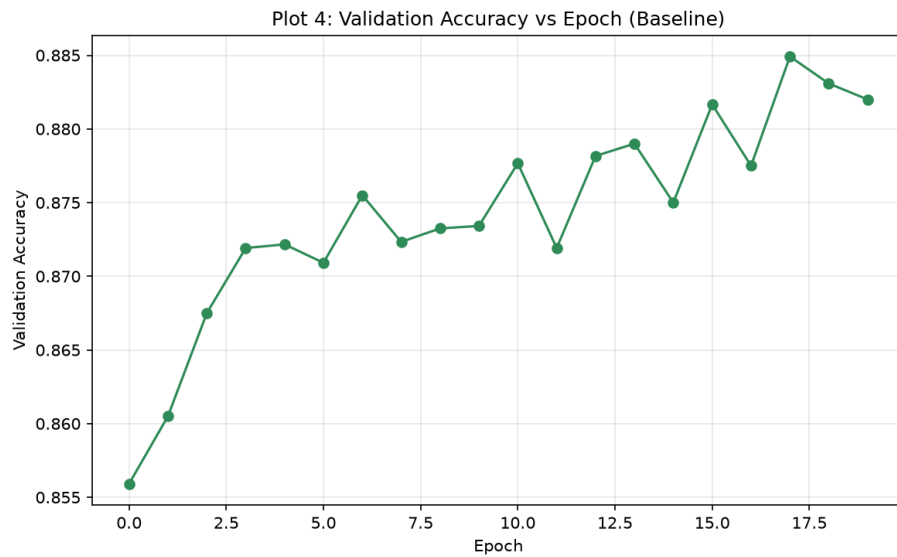


Figure 4: Validation accuracy vs. epoch (baseline model).

Inference: Validation accuracy improves from about 0.856 to a peak of 0.885 but has ups and downs across the epochs.

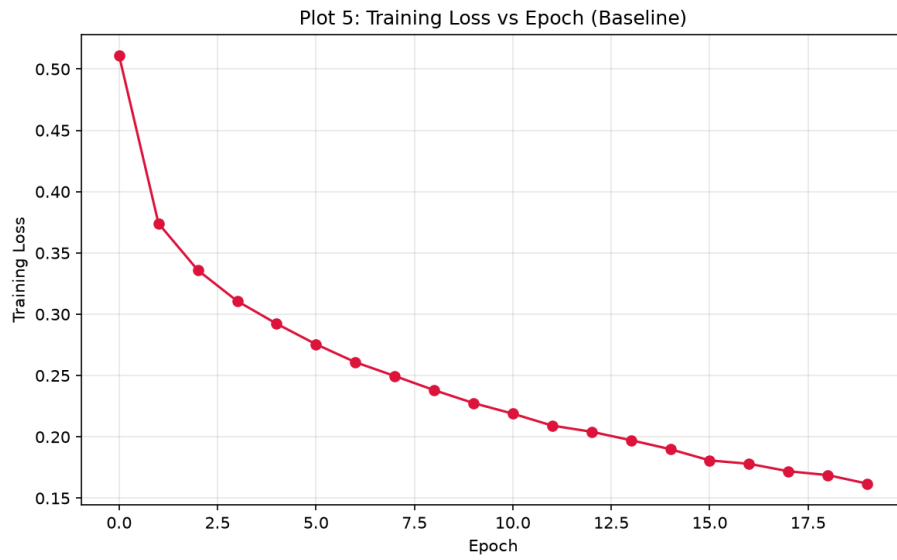


Figure 5: Training loss vs. epoch (baseline model).

Inference: Training loss decreases monotonically and smoothly from about 0.51 to 0.16, confirming stable convergence of the Adam optimizer on the categorical cross-entropy objective.

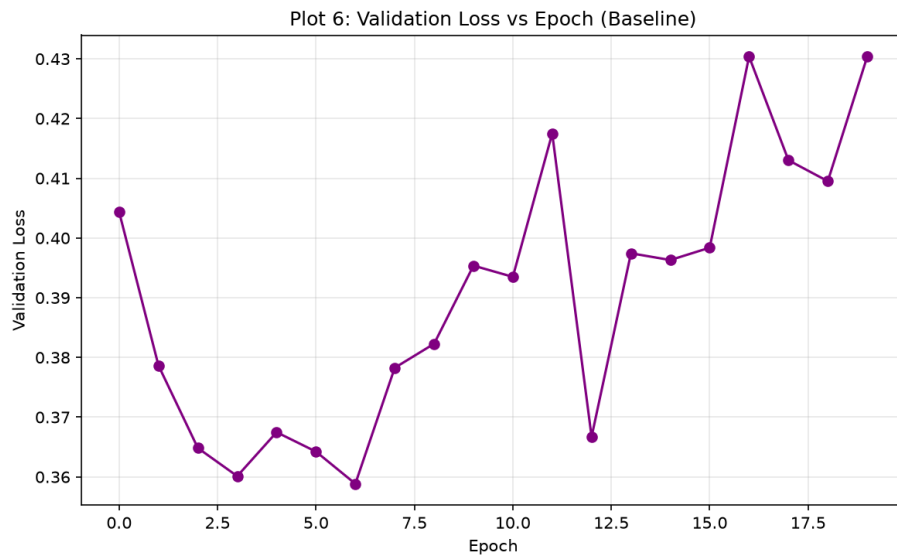


Figure 6: Validation loss vs. epoch (baseline model).

Inference: Validation loss decreases initially but rises after roughly epoch 6–7 even as training loss keeps falling, confirming the onset of overfitting in the later training epochs.

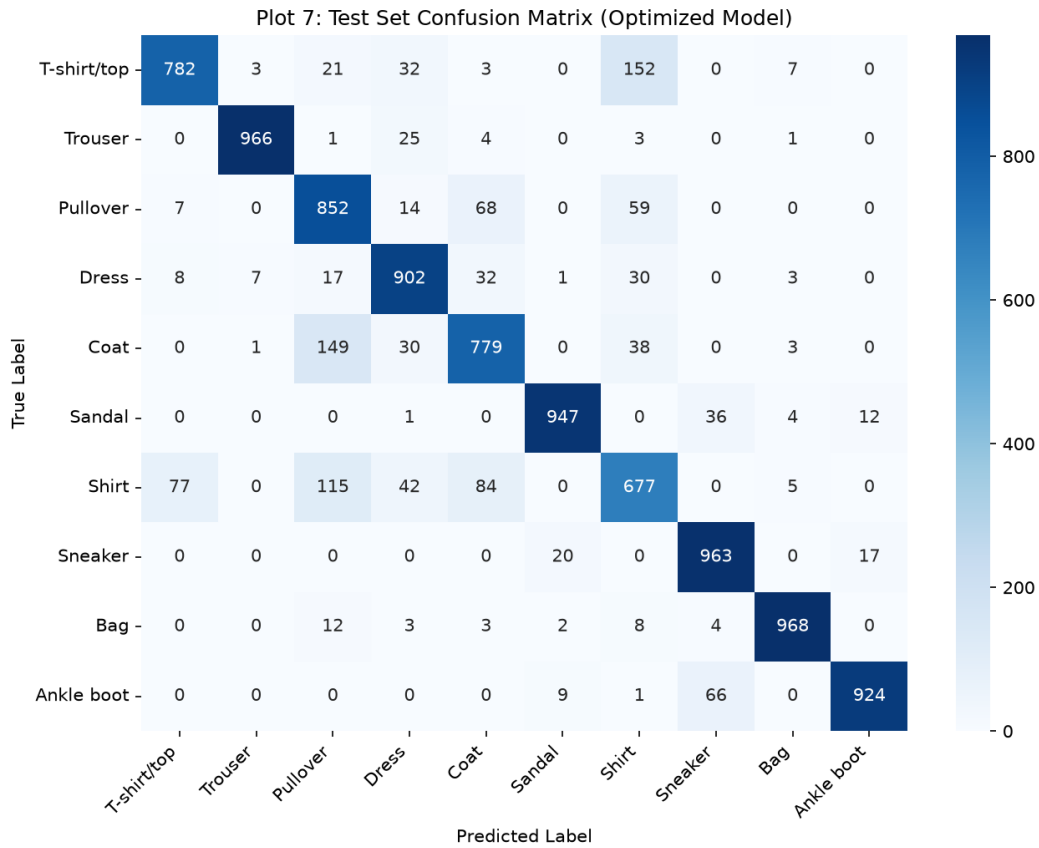


Figure 7: Test set confusion matrix (optimized model).

Inference: Most predictions concentrate along the diagonal, indicating strong per-class performance overall. The largest confusions occur between visually similar upper-body garments – notably Shirt being frequently confused with T-shirt/top (152) and Pullover with Coat (68) and Shirt (84).

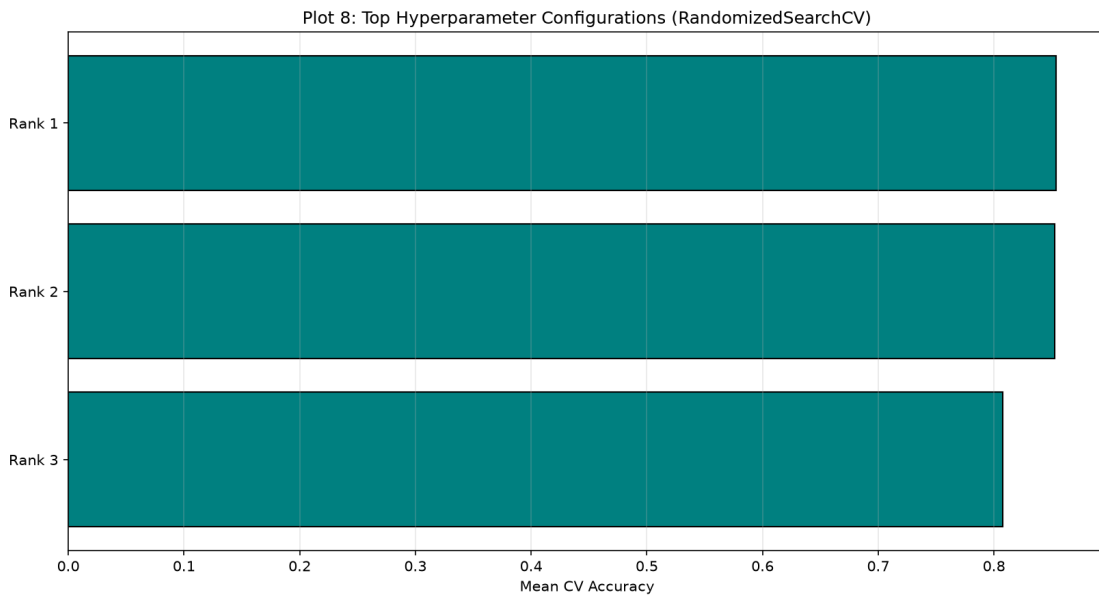


Figure 8: Top hyperparameter configurations from RandomizedSearchCV.

Inference: The top two sampled configurations (mean CV accuracy 0.8538 and 0.8526) perform similarly, while the third (sigmoid activation, higher learning rate) trails noticeably at 0.8077 – suggesting activation choice and learning rate strongly affect convergence quality within this limited sample.

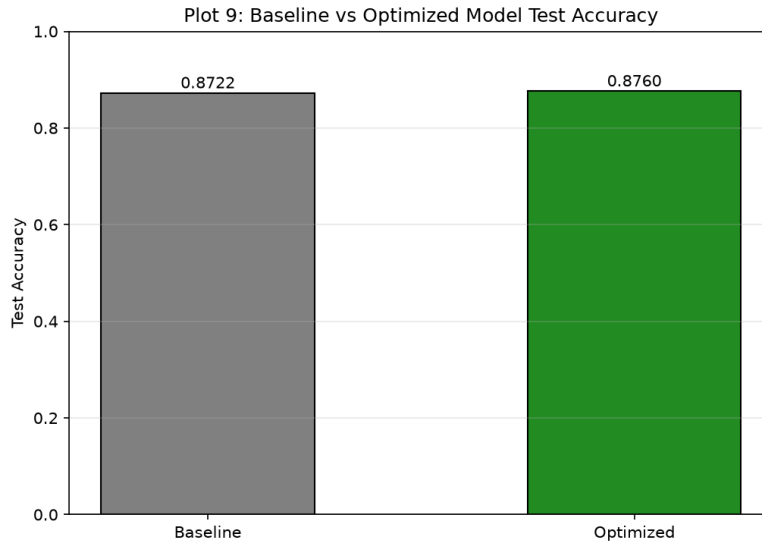


Figure 9: Baseline vs. optimized model test accuracy.

Inference: The optimized model achieves a modest test accuracy improvement over the baseline (0.8760 vs. 0.8722, +0.38 percentage points), suggesting the baseline architecture was already reasonably well-suited to this dataset.

9. Discussion

1. **Which optimization method was used?** RandomizedSearchCV with 5-fold cross-validation, using the SciKeras KerasClassifier wrapper.
2. **Which hyperparameters were selected?** 3 hidden layers of 128 neurons each, Tanh activation, RMSProp optimizer, learning rate 0.001, batch size 32, 30 training epochs, and a dropout rate of 0.2.
3. **Did optimization improve performance?** Yes, marginally. Test accuracy improved from 0.8722 (baseline) to 0.8760 (optimized), a gain of +0.0038. Macro F1-score similarly improved from 0.8725 to 0.8763.
4. **Which hyperparameter had the greatest impact?** Comparing the three sampled configurations, activation function and learning rate appear most influential: the two configurations using `tanh` activation with a learning rate of 0.001 scored 0.8538 and 0.8526, while the configuration using `sigmoid` activation with a higher learning rate of 0.01 scored markedly lower at 0.8077. This suggests sigmoid activation combined with a larger learning rate hinders convergence for this architecture, consistent with sigmoid’s known vanishing-gradient tendency in deeper networks.
5. **Compare Grid Search and Randomized Search.** Grid Search exhaustively evaluates every combination in the search space, guaranteeing the best combination within the grid, but its cost scales exponentially with the number of hyperparameters – exhaustively evaluating all 3,888 combinations at 5-fold CV would require 19,440 model fits, which is computationally intractable here. Randomized Search instead samples a fixed number of random combinations, trading exhaustiveness for tractable runtime. With only `n_iter=3` samples in this run, the search covered a very small fraction of the space, so the reported “best” configuration is best only among the samples drawn,

not necessarily globally optimal; a larger `n_iter` (e.g., 20–50) would give a more reliable estimate at the cost of longer runtime.

6. **Recommend the best MLP configuration.** Based on the (limited) search performed, the recommended configuration is 3 hidden layers of 128 neurons, Tanh activation, RMSProp optimizer, learning rate 0.001, batch size 32, 30 epochs, and dropout 0.2, which achieved the highest cross-validation accuracy (0.8538) and the highest test accuracy (0.8760) of the models evaluated.

10. Conclusion

An MLP was successfully implemented and trained on the Fashion-MNIST dataset using TensorFlow/Keras. The baseline architecture ($784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$), trained for 20 epochs with the Adam optimizer, achieved a test accuracy of 87.22%. Automated hyperparameter optimization via RandomizedSearchCV with 5-fold cross-validation identified a deeper 3-hidden-layer configuration using RMSProp and Tanh activation that improved test accuracy modestly to 87.60%. The confusion matrix and per-class metrics show that classification is strong overall, with most confusion occurring among visually similar upper-body garment classes (Shirt, T-shirt/top, Pullover, Coat) .

11. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.

12 Non-Linearly Separable Gate (XOR)

The XOR (Exclusive OR) gate evaluates to True (1) only when the inputs differ, and False (0) when they are the same. Plotted on a 2D plane, this creates a diagonal distribution of classes. The grid visually proves the mathematical limitations of a Single Layer Perceptron. Because it is geometrically impossible to draw a single straight line that separates the diagonal classes, the boundary oscillates wildly. Correcting a misclassified point on one side of the grid forces an error on the opposite side, trapping the algorithm in an infinite loop of weight updates and preventing convergence

XOR Gate Weight Evolution (Non-Convergence)

